

CC3501 - Tarea 1 - Otoño 2024

En esta tarea usted creará un programa que simule un *pinball* o *flipper*. La evaluación de este programa considera tres aspectos: 1) diseño; 2) vistas y proyecciones; 3) interactividad.

Diseño (3 puntos)

La cantidad de piezas en el diseño del flipper queda a su discreción, pero tiene por lo menos:

- Un tablero sobre el que se disponen los demás elementos.
- Dos *flippers* que son las paletas que se activan mediante *input* y que golpean a la pelota que se mueve por el tablero.
- Los bordes que limitan por dónde se podrá mover la pelota (el diseño queda a su discreción).
- Por lo menos 2 elementos con los que interactúa la pelota. Hay distintos tipos de interacción: chocar, rebotar, quedar atrapada y atravesar.
- Por lo menos 2 elementos decorativos que son visibles pero no son parte de la interacción con la pelota (por ej., en un *flipper* de *Star Trek* suele haber una nave espacial encima del tablero).

Puede ver un ejemplo de las piezas de un *flipper* aquí: [▶ Anatomy of a Pinball Machine](#) (active los subtítulos).

El puntaje de esta sección solo cuenta si los objetos son 3D:

- tablero → 0.5
- flipper → 0.5 c/u
- bordes → 0.5
- elementos extra → 0.25 c/u
- elementos decorativos → 0.25 c/u.

La nota máxima es 3 puntos, incluso si tiene más elementos de los pedidos.

Los elementos que no sean decorativos deben ser de su autoría. Puede hacerlos en el código o en *Blender* u otro software similar. Los elementos decorativos pueden ser modelos 3D descargados de la red.

Nota !: Todo lo que grafique deben ser modelos 3D. Se sugiere que haga diseños 2D que sean extendidos a 3D, como algunos de los elementos que se ven en la siguiente imagen:



Nota II: No es necesario que haga una bola en esta tarea. Su incorporación en el *flipper* es parte de la siguiente tarea. Sin embargo, debe implementar su tarea considerando que tendrá que hacer ese añadido más adelante. Para ello, asuma que la bola se desplazará en un mundo 2D, a pesar de que el despliegue gráfico es 3D (puede pensar como ejemplo un juego de plataformas “visto de lado” como *Super Mario* que, incluso si se grafica en 3D, es un mundo 2D).

Vistas y Proyecciones (1.5 puntos)

Usted debe implementar dos vistas distintas. Una es la vista superior del flipper, que utilizará una proyección ortográfica, y la otra es la vida “de jugador(a)”, que utiliza proyección en perspectiva. En ambas vistas procure que el tablero completo sea visible. El programa cambiará de vista al presionar la tecla C.

Interactividad (1.5 puntos)

Usted podrá interactuar con los flippers a través de las teclas A y D. Al presionar A, el flipper izquierdo se levantará; al presionar D, lo hará el derecho. La duración y amplitud del movimiento la decide usted.

¿Cómo hacer la tarea?

Esta es una sugerencia de estrategia para hacerla. No es la única manera, ni necesariamente la mejor, pero podría ayudarle a planificar la implementación. Siga los siguientes pasos:

1. Dibuje en un papel su diseño de flipper, de modo que conozca los tamaños y extensión de tablero y piezas.

2. Tener una **ventana de pyglet que dibuje un rectángulo y dos triángulos**. Ese rectángulo representará la superficie del *flipper* y los triángulos representarán las paletas. Recuerde que su *flipper* tiene piezas estáticas y piezas móviles, quizás sea bueno que tenga dos funciones, una que dibuje lo estático y otra lo dinámico.
3. Implemente las dos vistas utilizando lo visto en la clase de **vistas y proyecciones**. Puede hacerlo en el orden que prefiera. En su función de dibujo, al comienzo debiese evaluar cuál es la vista activa, y configurar sus *shaders* para esa vista.
4. Implemente la interactividad de las paletas utilizando **transformaciones**. Esto requerirá que usted lleve control del tiempo de la aplicación, de modo que sepa cuándo se activó la paleta. Por ejemplo, digamos que el movimiento de la paleta dura 1 segundo. Si usted presiona A, comience a contar el tiempo que ha pasado en la aplicación desde que se presionó la tecla. Ese tiempo t determinará el ángulo de transformación de la paleta. Una vez que se llega a $t = 1$ segundo, la paleta comienza a devolverse. Cuando vuelve a su punto de origen deja de estar en el estado “moviéndose” y no se le aplica ninguna transformación (o bien se le aplica una rotación de 0 grados, dependerá de cómo lo implemente).

Si usted hace lo anterior, tendrá un *flipper* mínimo y una tarea **funcional** que tiene un 4. Luego de eso puede trabajar en el diseño y objetos del *flipper* y así hacerlo tan bonito o psicodélico como usted desee para subir la nota.

Es importante que el *flipper* sea funcional porque será necesario para la siguiente tarea. Si hace la tarea al revés, enfocándose en el diseño visual, también puede tener nota 4, pero tendrá que ponerse al día con el resto de la funcionalidad para la tarea 2.

Ejecución

Suba los archivos [tarea1.py](#), [vertex_program.glsl](#) y [fragment_program.glsl](#) a U-Cursos. Su proyecto se ejecutará de la siguiente manera en la carpeta raíz del repositorio del curso:

```
$ conda activate grafica
$ python tarea1.py
```

Si utiliza modelos 3D o archivos con funciones auxiliares que no son parte del repositorio, debe subir todo el código en un archivo [.zip](#) que se descomprimirá en la carpeta raíz.

Si el programa se cae o la ventana no despliega un *flipper*, la nota es 1.

Fecha de entrega

19 de abril de 2024, 23:59 Hrs. No entregue en el último minuto, pues debido a motivos de carga es posible que U-Cursos no logre subir la tarea a tiempo.

No habrá plazo adicional, ni para subir tareas que no se alcanzaron a subir al sistema ni para dar el fin de semana. Por tanto, no deje la tarea ni su entrega para última hora¹.

Por favor: **¡NO DEJE LA TAREA PARA ÚLTIMA HORA!**

¹ Dato Rossa: de quienes han reprobado el curso en los últimos semestres, el motivo principal ha sido tareas y no controles. Recuerde que [tareas y controles se aprueban por separado](#).